

RoSC: Role-shared Semantic Coordination for Dynamic Heterogeneous Multi-arm Manipulation

Abstract—Heterogeneous multi-arm manipulation can combine complementary workspaces, end-effectors, and sensing capabilities for concurrent task execution in dynamic workspaces. However, semantic coordination becomes difficult when the physical scene changes continually while task-relevant updates, external disturbances, and execution failures occur only at discrete moments. Static or monolithic semantic coordination can either miss these events or trigger unnecessary global replanning that interrupts still-valid execution. This work addresses this problem through role-shared semantic coordination for dynamic heterogeneous multi-manipulator scenarios. The framework assigns explicit semantic responsibilities to the affordance, planning, and verification roles over a shared semantic state that stores the current instruction, the assignment-aware primitive graph, task-relevant scene facts, and execution status. An event-driven selective semantic update procedure further identifies the affected scope caused by a task update, external disturbance, or execution failure, and revises only that scope while preserving unaffected valid execution. Simulation baselines, failure analysis, and ablations show that explicit responsibility separation reduces coordination errors and selective semantic updates enable faster adaptation in evolving environments; hardware trials examine the same mechanisms on a heterogeneous multi-arm platform.

I. INTRODUCTION

Heterogeneous multi-manipulator systems must maintain a consistent task interpretation while complementary robots execute concurrently in a changing workspace. Figure 1 instantiates this problem with a conveyor and three task-relevant semantic events. Language- and vision-language-based grounding [1], [2], [3] does not expose team capability constraints or responsibility interfaces; independent agents can therefore maintain incompatible task views. The challenge is to establish one capability-aware semantic authority across robots and revise only the semantics invalidated by a new instruction, disturbance, or verified failure while local controllers remain responsive to continuous scene evolution.

A. Related Work

Language-guided grounding methods translate natural-language instructions into spatial representations, executable programs, or scene-grounded task plans [1], [2], [3], [4]. Building on these capabilities, LLM-based multi-robot frameworks adopt different coordination structures: RoCo uses inter-robot dialogue; COHERENT relies on centralized proposals and execution feedback; MALLVI reactivates specialized agents when needed; and CoMuRoS and DEXTER-LLM revise tasks or assignments in response to events [5], [6], [7], [8], [9]. Other systems explicitly incorporate capability-aware coalitions, dependency-based task allocation, or separate planning, manipulation, and verification

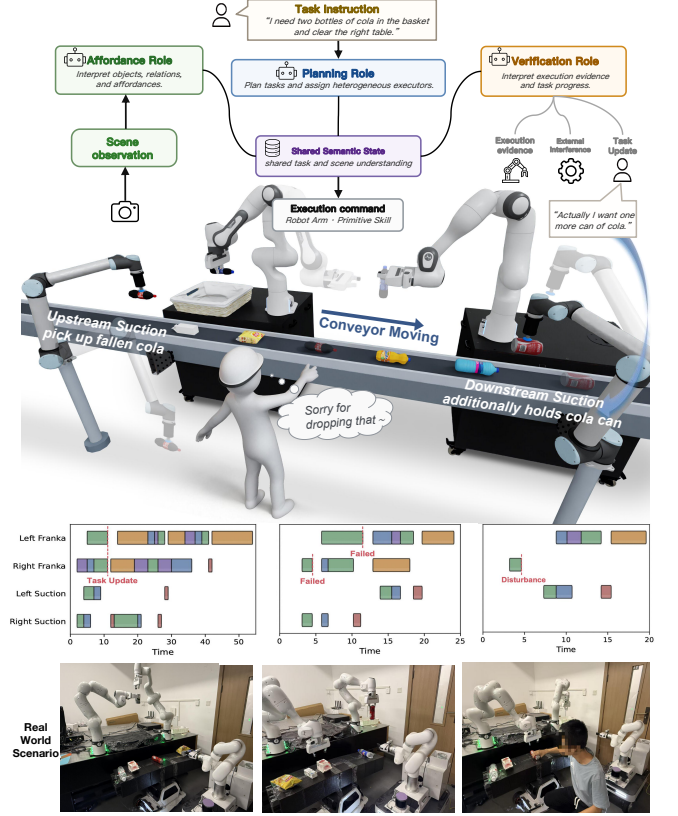


Fig. 1. The shared semantic state coordinates three roles while preserving local execution. **Top:** framework and simulated conveyor workspace; **Middle:** event Gantt charts; **Bottom-left:** normal hardware execution; **Bottom-middle:** online task update; **Bottom-right:** disturbance by human.

agents [10], [11], [12]. Collectively, these methods establish effective mechanisms for grounding, agent interaction, and task revision, but they do not explicitly organize ownership of the distinct semantic components that must remain consistent as multiple heterogeneous manipulators execute concurrently. RoSC addresses this issue by maintaining the authoritative shared state and assigning responsibility to the affordance, planning, and verification roles, respectively. When a discrete semantic event occurs, these roles identify its dependency-closed affected scope and reconstruct only the invalidated subgraph, while preserving unaffected valid execution and leaving continuous physical adaptation to the local controllers.

Dynamic manipulation supports tracked or reactive grasping and visual servoing [13], [14], [15], [16], allowing local controllers to continuously adapt end-effector motion to moving targets and perception updates. Execution monitoring and continual replanning further address runtime deviations

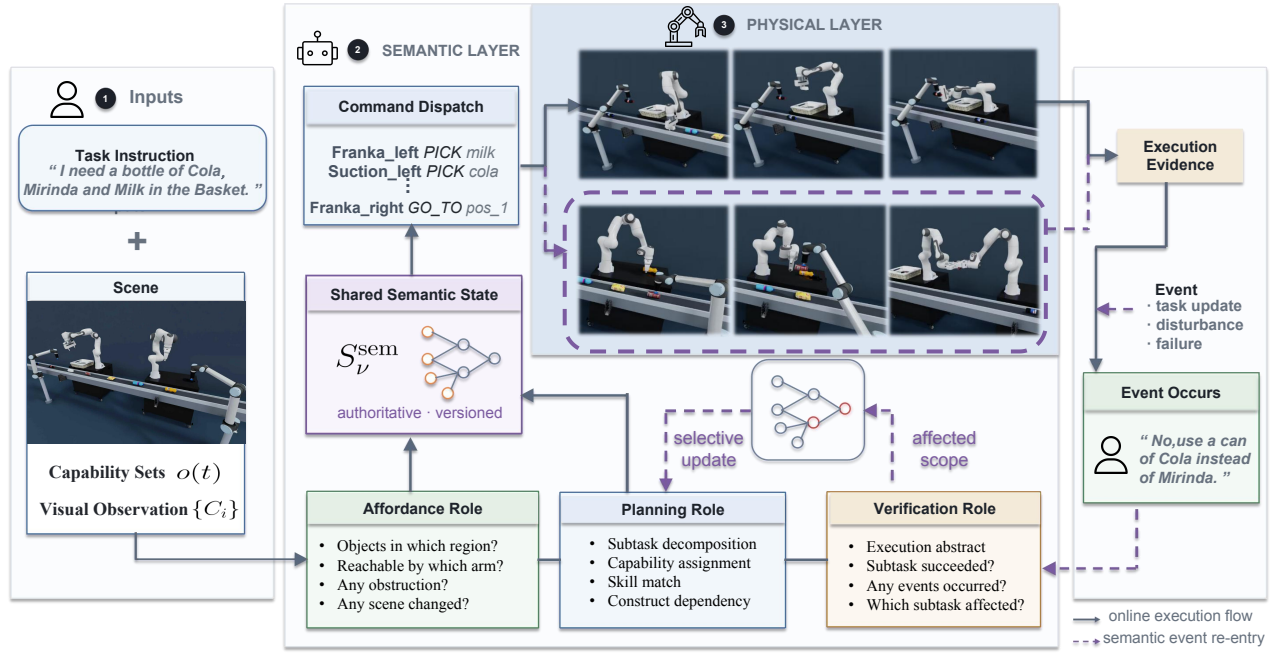


Fig. 2. Role-shared coordination links semantic decisions to physical execution. Affordance evaluates scene feasibility; planning decomposes and assigns subtasks, skills, and dependencies; verification abstracts execution evidence, checks outcomes and events, and identifies affected subtasks. Solid and dashed arrows denote online execution and selective event updates.

by detecting discrepancies between expected and observed outcomes and invoking recovery or plan revision [17], [18], [19], [20]. These methods primarily focus on maintaining physical feasibility, monitoring action execution, or revising a plan after deviations. However, in heterogeneous multi-manipulator settings, the resulting feedback may affect different parts of the team-level semantic state: a new observation may change an object’s affordance, a failed action may invalidate its execution status and downstream dependencies, and a revised instruction may alter only one task branch. RoSC explicitly separates these responsibilities among affordance, verification, and planning roles. It uses the shared semantic state to determine which information must be refreshed, which subtasks are affected, and whether planning should be invoked, thereby supporting event-scoped semantic revision while preserving unaffected valid execution.

B. Our Method

We propose RoSC, a role-shared semantic coordination framework with an event-driven selective update that follows event detection $\rightarrow$ affected-scope selection $\rightarrow$ semantic-state revision $\rightarrow$ continued dispatch from a consistent frontier. Simulation baselines, failure analysis, and mechanism ablations test these mechanisms, while hardware trials check the same update sequence during physical execution. As shown in Figure 2, the affordance role evaluates object regions, reachability, obstructions, and scene changes to produce $\mathcal{F}$; the planning role decomposes the task, matches skills and capabilities to manipulators, constructs G , and initializes the versioned shared semantic state $S_\nu^{\text{sem}} = (\gamma, G, \mathcal{F}, z)$. The framework then selects the verified executable frontier, constructs capability-valid commands, and dispatches independent commands concurrently to the assigned manipulators.

After local execution, affordance refreshes $\mathcal{F}$ and verification abstracts the evidence, checks the outcome, updates z , and detects events. For an event, verification identifies $\mathcal{N}_k^{\text{aff}}$; planning replans that subgraph while preserving valid work outside it, then dispatch continues from the updated frontier. This restricted semantic update shortens the response path and helps preserve opportunities as the scene evolves.

The main contribution of this work is twofold: (I) A role-shared semantic coordination formulation is introduced for dynamic heterogeneous multi-manipulator scenarios with continual physical changes, assigning explicit responsibility to the affordance, planning, and verification roles; and (II) An event-driven selective semantic update procedure is developed to efficiently handle task updates, external disturbances, and execution failures while preserving valid execution.

II. PROBLEM FORMULATION

A. Heterogeneous Manipulator Team

Consider a team of heterogeneous robotic manipulators

$$\mathcal{R} \triangleq \{r_i\}_{i=1}^N, \quad (1)$$

where the team in (1) contains N manipulators and r_i has capability set $\mathcal{C}_i$. A capability set describes the primitive actions, targets, and embodiment conditions available to the corresponding manipulator. The system receives a natural-language task γ and continuous visual observations $o(t)$ while the physical workspace evolves. The execution process can also produce discrete semantic events

$$e_k \in \mathcal{E} \triangleq \{e^t, e^x, e^f\}, \quad (2)$$

where e^t denotes an online task update, e^x denotes an external disturbance, and e^f denotes an execution failure.

Continuous physical motion, including motion of a target on a conveyor, remains part of the evolving workspace and is not itself a semantic event.

B. Online Primitive Command Sets

An executable command c is an instruction that can be sent directly to a manipulator execution interface. It specifies, through the interface representation rather than a semantic graph node, the execution skill, semantic target or object, and manipulator that executes the skill. Thus, a semantic subtask or graph node is not itself a command; the required output is an online sequence of command sets:

$$\mathbf{C} \triangleq (\mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_M), \quad (3)$$

where M is determined online, each $\mathbf{c}_m$ contains capability-valid commands that can be issued concurrently to the team, and the internal command fields are defined in Section III-C.1. The next command set is generated online by the role-shared coordination pipeline from the task, heterogeneous capabilities, visual history, semantic event history, and previously issued command sets:

$$\mathbf{c}_m = \Pi_{\text{coord}}(\gamma, \{\mathcal{C}_i\}_{i=1}^N, o(0:t_m), e_{1:k_m}, \mathbf{c}_{1:m-1}), \quad (4)$$

where t_m is the decision time and k_m is the number of semantic events observed by that time. The operator Π_{coord} in (4) denotes the complete online pipeline whose internal stages construct the dependency graph G , executable commands, and verification status as defined in the Method. The ordering of the arguments preserves the information flow from task specification and embodiment capabilities to observations, events, and prior command sets.

C. Online Coordination Problem

The physical workspace may change continuously between decision times, and the event stream may contain task updates, external disturbances, or execution failures. The command sequence must support concurrent progress while respecting each manipulator's capability set and the dependencies induced by the task. No scalar optimization objective is imposed in this formulation; the primary requirement is completion of the current task under the stated online information and execution conditions.

Problem 1: Given a natural-language task γ , a heterogeneous team $\mathcal{R}$ with capability sets $\{\mathcal{C}_i\}_{i=1}^N$, continuous visual observations $o(t)$, and the event history in (2), compute the online command-set sequence $\mathbf{C}$ in (3) such that the primitive commands are capability-valid, independent commands can execute concurrently, and the current task is completed despite continuous physical evolution and the possible occurrence of e^t , e^x , or e^f . ■

III. METHOD

This section presents the role-shared semantic coordination framework shown in Figure 2. Its three roles respectively evaluate scene feasibility, decompose and assign task dependencies, and verify outcomes while identifying event-affected subtasks. Section III-A defines the shared semantic

state and these three responsibilities, Section III-B describes event-scoped updates, Section III-C constructs and verifies executable commands, and Section III-D gives the online algorithm and complexity discussion.

A. Role-Shared Semantic State and Responsibilities

The shared semantic state is the authoritative task-level state maintained by the three semantic roles during the current coordination cycle:

$$S_\nu^{\text{sem}} \triangleq (\gamma, G, \mathcal{F}, z), \quad (5)$$

where ν is the semantic version, γ is the current task instruction, G is the assignment-aware primitive graph, $\mathcal{F}$ contains task-relevant scene and feasibility facts, and z records primitive-subtask execution status. This state excludes manipulator-local control variables and stores only the semantic information used for task-level coordination.

1) *Scene Grounding and Feasibility Assessment:* The affordance role maintains $\mathcal{F}$ through

$$\rho_A : o(t) \mapsto \mathcal{F},$$

which extracts task-relevant scene and feasibility facts from the observation stream, including object grounding, relations, affordances, and feasible primitive interactions. During initialization, observed objects are grounded to semantic targets in γ . After each primitive execution, the latest observation refreshes $\mathcal{F}$ before planning or verification consumes the updated physical facts.

2) *Task Decomposition and Capability Assignment:* The planning role maintains G by first deriving an intermediate abstract dependency graph from γ and $\mathcal{F}$, then adding manipulator assignment and execution dependencies. The final assignment-aware primitive graph is given by:

$$G = \rho_P(\gamma, \mathcal{F}, \{\mathcal{C}_i\}_{i=1}^N, \mathcal{R}) \triangleq (\mathcal{N}, \mathcal{L}), \quad (6)$$

where for each $n \in \mathcal{N}$ $n \triangleq (s_n, q_n, r_i, \kappa_n)$ denotes an assigned primitive node with primitive skill type s_n , semantic target q_n , assigned manipulator r_i , and execution skill κ_n . The relations in $\mathcal{L}$ include the semantic dependencies from the dependency graph and manipulator-aware dependencies by $\mathcal{R}$. Thus, G records what to execute, who executes it, and which skill is used. For $n \in \mathcal{N}$, $\text{pred}(n)$ denotes the predecessor j such that $(j, n) \in \mathcal{L}$.

3) *Execution Assessment and Status Tracking:* The verification role maintains z after command execution. For an executable command c_n , the command sent to the assigned manipulator interface specifies the required execution skill and semantic target associated with primitive subtask n . After c_n is executed, the latest observation first refreshes $\mathcal{F}$; the verification role then extracts transient evidence:

$$\eta_n = \rho_V^\epsilon(c_n, \mathcal{F}), \quad (7)$$

where η_n is the transient execution evidence for the executed command. The verification role compares evidence with the current instruction to update status and detect new events:

$$(e_k, z(n)) = \rho_V^z(\eta_n, \gamma), \quad (8)$$

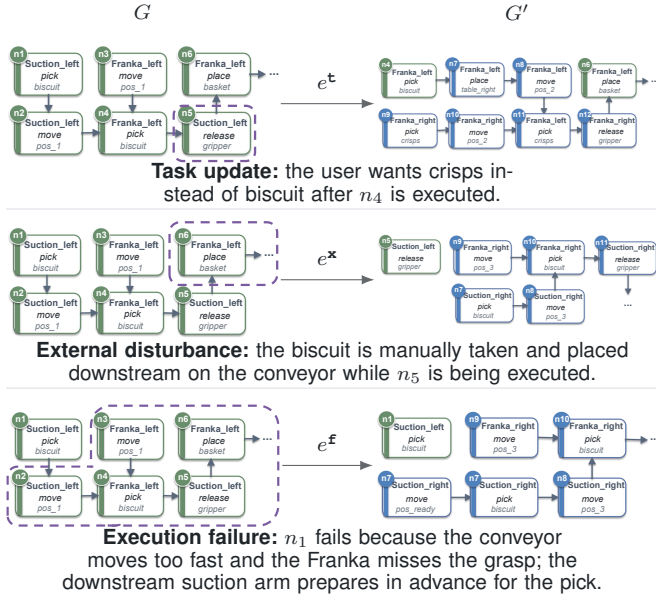


Fig. 3. Selective graph transitions $G \rightarrow e_k \rightarrow G'$ for task updates, external disturbances, and execution failures. Purple contours mark $\mathcal{N}_k^{\text{aff}}$, unfinished or dependency-invalidated nodes requiring reconsideration and reconstruction; failed attempts recorded in z need not be reconstructed. Green nodes are retained and blue nodes are new.

where $z(n)$ is the primitive-subtask status and e_k records any detected event in (2); its affected scope is identified in Section III-B. Without a task-relevant inconsistency, e_k is null and only $z(n)$ is updated. Within $S_\nu^{\text{sem}} = (\gamma, G, \mathcal{F}, z)$, the affordance, planning, and verification roles own $\mathcal{F}$, G , and z , respectively. An updated γ enters as an external semantic constraint and is committed before role-specific updates. The intermediate graph is used only to construct G , and η_n remains transient verification evidence.

B. Event-driven Selective Semantic Updates

Task-relevant events enter the shared semantic state, whereas continuous physical evolution remains within manipulator-local execution. Event handling identifies the invalid scope and invokes adaptation, as shown in Fig. 3.

1) *Event-affected Scope Identification:* The event types are defined in (2). A task update changes the current instruction, an external disturbance is interpreted from fresh visual evidence, and an execution failure is established by verification. The event then enters the verification role, which identifies the affected assignment-aware primitive nodes:

$$\mathcal{N}_k^{\text{aff}} \triangleq \rho_V^{\text{aff}}(e_k, \gamma, G, \mathcal{F}, z), \quad (9)$$

where $\mathcal{N}_k^{\text{aff}} \subseteq \mathcal{N}$ contains the nodes whose task meaning, feasibility, execution status, or dependencies require reconsideration. The operator tests e_k against the current γ , G , refreshed $\mathcal{F}$, and verified z . Its event-specific inconsistency checks select requirements invalidated by a task update, scene-dependent assumptions invalidated by a disturbance, or nodes marked FAILED after execution. Verification then traces outgoing dependencies in G to include unfinished descendants whose validity or executability depends on those

nodes, while independent valid nodes remain outside $\mathcal{N}_k^{\text{aff}}$ and are preserved.

2) *Selective Semantic-state Reconstruction:* The selective update produces the next committed semantic state:

$$S_{\nu+1}^{\text{sem}} \triangleq \mathcal{U}(S_\nu^{\text{sem}}, e_k, \mathcal{N}_k^{\text{aff}}), \quad (10)$$

where $\mathcal{U}$ denotes the final result of the coordinated role updates rather than a separate replanning step. Before this result is committed, the affordance role refreshes $\mathcal{F}$, and the verification role updates z and produces $\mathcal{N}_k^{\text{aff}}$ by (9). Using this affected set and the latest $\mathcal{F}$, the planning role retains exactly the nodes of G outside $\mathcal{N}_k^{\text{aff}}$, together with the relations whose endpoints are both retained. It replans the nodes in $\mathcal{N}_k^{\text{aff}}$, grounds and capability-assigns their replacements from the updated state, and reconnects the revised subgraph to the retained graph at the dependency boundary, thereby completing a partial reconstruction of G . The status of each retained node is preserved, whereas each new or revised node starts as PENDING. The reconstructed graph is checked for capability-valid assignments and predecessor consistency. The affordance, planning, and verification roles have then respectively updated $\mathcal{F}$, G , and z ; together with the event-consistent γ , these components form $S_{\nu+1}^{\text{sem}}$ and advance the semantic version from ν to $\nu + 1$.

C. Primitive-command Dispatch and Execution

1) *Capability-valid Command Construction:* The command notion is instantiated after planning has produced assignment-aware primitive nodes. At each execution step, $\mathcal{N}_{\text{exec}}$ is the temporary set of executable primitive subtasks in G . A node n belongs to $\mathcal{N}_{\text{exec}}$ when $z(n) \neq \text{COMPLETE}$ and $z(n') = \text{COMPLETE}$ for every $n' \in \text{pred}(n)$. For any $n \in \mathcal{N}_{\text{exec}}$, command construction is given by:

$$c_n = \phi_C(n), \quad \phi_C : \mathcal{N}_{\text{exec}} \rightarrow \mathcal{C}, \quad (11)$$

where ϕ_C is the command construction function and $\mathcal{C}$ denotes the executable command space, distinct from the per-manipulator capability sets $\{\mathcal{C}_i\}_{i=1}^N$. Although n contains semantic execution information, c_n is the executable command sent to the robot interface. At each execution step, the commands generated from $\mathcal{N}_{\text{exec}}$ form a concurrent command set $\mathbf{c}_m = \{c_{n_1}, c_{n_2}, \dots\}$.

2) *Local Execution and Outcome Verification:* The assigned local skill executes each command using the manipulator's current perception, tracking, and control stack. Its target reference allows continuous motion to remain local. After execution, the latest observation refreshes $\mathcal{F}$ through $\mathcal{F} \leftarrow \rho_A(o'_t)$ without semantic replanning. For each executed command, verification extracts η_n via (7) and uses (η_n, γ) to update $z(n)$ and detect any e_k via (8). A detected event triggers identification and selective state revision.

D. Closed-Loop Coordination

1) *Online Coordination Algorithm:* Algorithm 1 summarizes the online procedure. At task start, the affordance role observes the scene and extracts $\mathcal{F}$. The planning role then derives the task dependencies, constructs the assignment-aware

Algorithm 1: Online role-shared semantic coord.

Input: $\gamma, o(t), \{\mathcal{C}_i\}_{i=1}^N, \mathcal{R}$
Output: $\mathbf{C}$

- 1 Extract $\mathcal{F}$ from $o(0)$;
- 2 Construct G by (6);
- 3 Initialize S_ν^{sem} by (5); $\mathbf{C} \leftarrow ()$;
- 4 **while** task incomplete **do**
- 5 $\mathcal{N}_{\text{exec}} \leftarrow \{n \in \mathcal{N} \mid z(n) \neq \text{COMPLETE}, z(n') = \text{COMPLETE}, \forall n' \in \text{pred}(n)\}$;
- 6 $\mathbf{c}_m \leftarrow \{c_n \mid n \in \mathcal{N}_{\text{exec}}\}$ using (11);
- 7 Append $\mathbf{c}_m$ to $\mathbf{C}$ and execute it;
- 8 Refresh $\mathcal{F}$ from o_t ;
- 9 **foreach** $c_n \in \mathbf{c}_m$ **do**
- 10 Extract η_n by (7);
- 11 Update $(e_k, z(n))$ by (8);
- 12 **if** e_k is detected **then**
- 13 Identify $\mathcal{N}_k^{\text{aff}}$ by (9);
- 14 Retain $G \setminus \mathcal{N}_k^{\text{aff}}$ and reconstruct the affected subgraph using $\mathcal{F}$;
- 15 Commit $S_{\nu+1}^{\text{sem}}$ by (10); $\nu \leftarrow \nu + 1$;

G , and initializes $S_\nu^{\text{sem}} = (\gamma, G, \mathcal{F}, z)$. During each loop, the verified frontier selects independent executable nodes, maps them to capability-valid commands, and dispatches the concurrent command set to the assigned manipulators. After local execution, affordance refreshes $\mathcal{F}$; for every executed command, verification extracts η_n and uses (η_n, γ) to update $z(n)$ and detect any e_k . When an event occurs, verification reads the refreshed $\mathcal{F}$ together with e_k, γ, G , and z , applies the event-specific criterion and traces unfinished dependent nodes, and outputs $\mathcal{N}_k^{\text{aff}}$. The planning role then retains the nodes outside this set, replans only the affected nodes using the latest $\mathcal{F}$, and reconnects the revised subgraph. The three role updates are committed as $S_{\nu+1}^{\text{sem}}$, after which dispatch resumes from the new consistent frontier. Restricting recomputation to this dependency-closed scope is intended to shorten semantic-event response time and reduce the chance that ongoing physical evolution removes an available manipulation opportunity.

2) *Event-specific Coordination:* The same protocol handles all events. For E1, verification marks the requirement-changing branch and planning regenerates only that branch. For E2, refreshed $\mathcal{F}$ invalidates nodes whose target, feasibility, or dependency assumptions changed. For E3, the failed node and unfinished downstream descendants receive a repair branch. Valid completed nodes, independent pending work, and local tracking are retained. Thus, affordance supplies current facts, verification selects scope, and planning revises only the invalidated semantic subgraph.

3) *Computational Complexity:* The explicit graph operations in Algorithm 1 are linear in the maintained graph when predecessor lists are stored with the graph. Checking executable nodes requires scanning $\mathcal{N}$ and the relevant incoming dependencies, with worst-case cost $O(|\mathcal{N}| + |\mathcal{L}|)$ per dispatch step. After an event, selective revision is bounded by the affected subgraph identified by $\mathcal{N}_k^{\text{aff}}$ and its incident

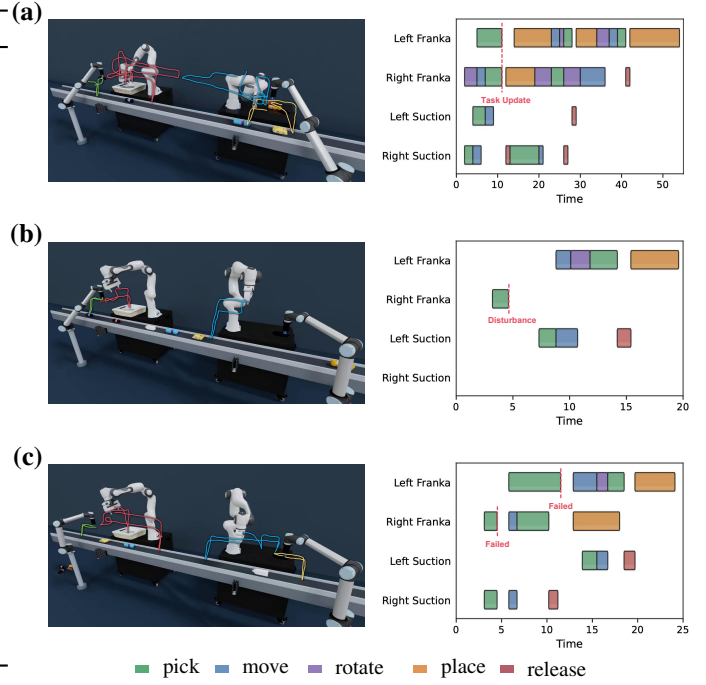


Fig. 4. Paired trajectories and execution timelines expose the localized response to each semantic event. (a) Task Update: the request changes to replace Mirinda with cola. (b) External Disturbance: the cola is moved to the other end of the conveyor during grasping. (c) Execution Failure: the conveyor moves too fast and both Franka manipulators fail their picks in succession. In each row, the left panel is the manipulator trajectory and the right panel is the corresponding Gantt chart.

dependencies. The complexity of the semantic operators ρ_P , ρ_V^e , ρ_V^z , and ρ_V^{aff} also includes the call time of the concrete LLM/VLM models used by these operators. Because these call times depend on the selected models and serving conditions, no closed-form bound is assigned here.

IV. EXPERIMENTS

This section tests whether role-shared semantic coordination improves task correctness, preserves useful concurrent execution, and limits event-driven updates to invalidated work. Simulation compares the proposed method with three matched baselines over static, dynamic, and event conditions. Failure counts and mechanism ablations then localize the source of the differences. Hardware trials provide a physical check of the same semantic-state update sequence.

A. Experimental Setup

The simulation uses the Isaac conveyor workspace in Fig. 1. It contains heterogeneous manipulators, static and moving targets, global visual observations, and a common primitive-skill interface. All methods use the same scene and task assets, observations, success criterion, tracking and safety stacks, and physical execution interface. Semantic calls use the LLM deepseek-ai/DeepSeek-V4-Flash and the VLM Qwen/Qwen3.5-397B-A17B. Conditions C1-C4 cover static and dynamic execution with task-relevant events E1 (task update), E2 (external disturbance), and E3 (verified execution failure). Each simulation condition contains 30

TABLE I
BASELINE TASK SUCCESS RATE (TSR) OVER 30 TRIALS.

Method	C1	C2			C3	C4		
		E1	E2	E3		E1	E2	E3
Ours	30/30	30/30	30/30	30/30	27/30	30/30	30/30	27/30
End2End	30/30	10/30	27/30	3/30	0/30	0/30	3/30	13/30
Agent per Robot	10/30	3/30	13/30	17/30	3/30	0/30	0/30	0/30
COHERENT	30/30	23/30	30/30	20/30	30/30	23/30	27/30	17/30

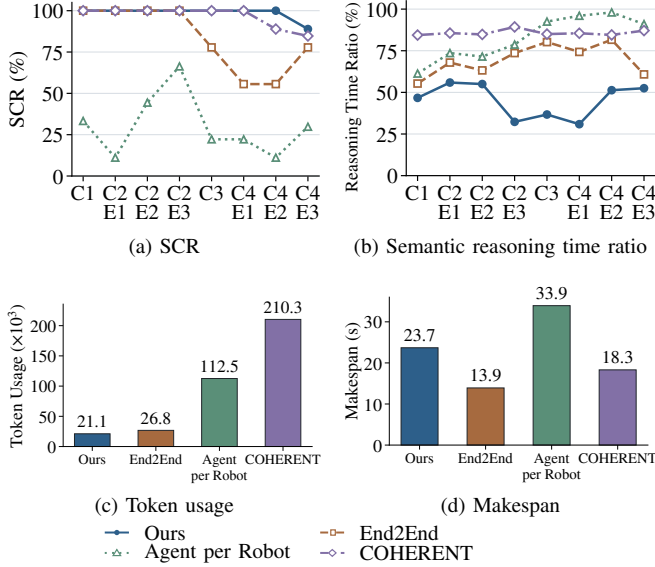


Fig. 5. Secondary event-handling metrics: (a) SCR, (b) semantic reasoning time ratio, (c) token usage, and (d) successful-trial makespan.

independent trials with randomly sampled initial object layouts; events are injected at 30%–70% of subtask-completion progress. Evaluation uses task success, completion, efficiency, semantic reasoning, and update-related metrics summarized in Tables I–III and the associated figures. Hardware conditions use 10 trials.

B. Main Results

The proposed loop maintained high completion across all tested regimes while changing only the event-relevant portion of the shared state. TSR was 30/30 in the static, task-update, and disturbance conditions and 27/30 in the two execution-failure conditions; SCR was 100% except for 88.9% in C4–E3. The unsuccessful trials therefore occurred only under verified execution-failure conditions rather than ordinary motion or task updates. Figure 3 tracks the corresponding semantic-state transitions. A task update revises γ and the affected branches of G ; an external disturbance refreshes $\mathcal{F}$ and reconstructs branches whose stored assumptions no longer hold; and an execution failure recorded in z triggers repair of the failed node and its unfinished dependencies. In each case, $\mathcal{N}_k^{\text{aff}}$ is identified, unaffected valid nodes are retained, and only the affected subgraph is reconstructed, providing qualitative evidence of selective update.

The paired trajectories and timelines in Fig. 4 show how this design is realized. Verified independent nodes are dispatched together, local tracking handles continuous

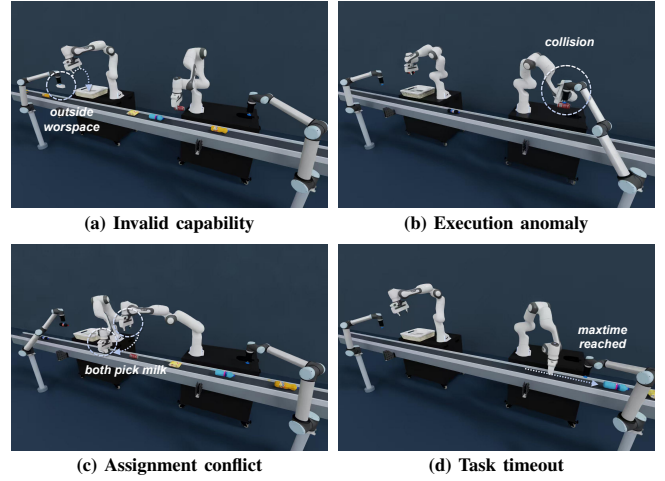


Fig. 6. Representative failures: (a) invalid workspace capability, (b) collision, (c) conflicting assignment, and (d) timeout after repeated replanning.

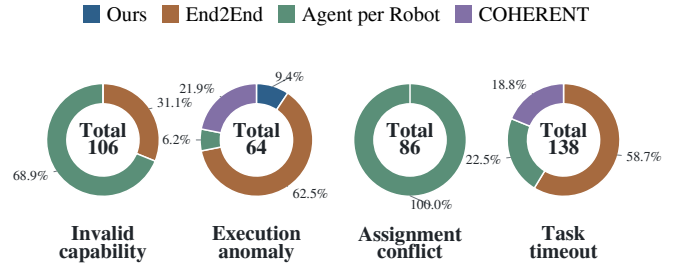


Fig. 7. Failure counts and baseline proportions by failure type.

target motion, and semantic reasoning is invoked only at initialization, verification boundaries, or task-relevant events. The reported time, makespan, and token ranges quantify this trade-off; the larger token cost in C4–E3 is consistent with its additional verified repair branch.

C. Comparison with Baselines

The comparison uses the same task, observations, capabilities, primitive skills, trial count, and success criterion for every method. **End2End** directly maps the task and observation to commands without the shared semantic state. **Agent per Robot** gives each manipulator an independent language-model agent, whereas **COHERENT** adapts the heterogeneous multi-robot collaboration protocol of Liu *et al.* [6] to the same interface.

Ours retained 30/30 on all task-update and disturbance cases and 27/30 under execution failure, while every baseline lost substantially more event trials. Table I reports their condition-wise TSR, making the event-specific gap below directly comparable. The largest gap occurred when semantic events had to be incorporated. In static C1, all methods except Agent per Robot obtained 30/30, isolating the difference to event-aware coordination rather than the basic skill interface. The secondary metrics in Fig. 5 show the same ordering: among the compared methods, Ours had the highest event-condition SCR and the lowest semantic-time ratio and token use. Ours had a longer successful-trial makespan than End2End but higher event-condition completion.

(a) H1 Normal cooperative execution



(b) H2 Task update adaptation



(c) H3 External disturbance recovery



Fig. 8. Hardware sequences provide verification evidence for normal motion, task update, and external disturbance.

TABLE II

ABLATION TSR, SCR AND REASONING TIME UNDER C4 (30 TRIALS).

Method	E1			E2			E3		
	TSR	SCR	Time	TSR	SCR	Time	TSR	SCR	Time
Ours	30/30	100	.309	30/30	100	.513	27/30	88.9	.525
w/o Role Separation	23/30	82.1	.462	30/30	100	.567	0/30	78.2	.796
w/o Selective Replanning	26/30	100	.456	30/30	100	.545	17/30	93.0	.658

	Invalid capability	Execution anomaly	Assignment conflict	Task timeout
Ours	0	3	0	0
A1	9	10	0	18
A2	0	13	0	4

Fig. 9. Ablation failure counts under C4. A1 and A2 denote the variants without role separation and selective replanning, respectively.

D. Failure Analysis and Ablation Study

1) *Cause of Task Failures:* The failure counts in Fig. 7 quantify the intermediate coordination state. Ours had no capability, assignment, or timeout failures and only six physical execution anomalies, whereas the baselines exhibited the corresponding semantic failures. Fig. 6 shows representative cases: role ownership filters invalid dispatches and verification exposes residual physical anomalies for repair.

2) *Key Ablations:* Under C4, the ablations remove role ownership or selective replanning. Table II shows lower event TSR for both, most under execution failure. Role-ablation errors in Fig. 9 support this attribution: the role-separated design limits error propagation by assigning each updated semantic component a dedicated owner, namely affordance for $\mathcal{F}$, planning for G , and verification for z . Without selective replanning, higher semantic-time ratios show that semantic reasoning occupies more of the execution horizon. Together

TABLE III

HARDWARE RESULTS OVER 10 TRIALS. RESP. IS EVENT-RESPONSE TIME (S); AFF./PEND. IS THE AFFECTED-TO-PENDING NODE RATIO.

Condition	TSR	SCR (%)	Resp. (s)	Aff./Pend.
H1 Normal	10/10	100.0	—	—
H2 Task Update	9/10	98.6	11.3	0.38
H3 External Disturbance	7/10	92.3	9.7	0.14

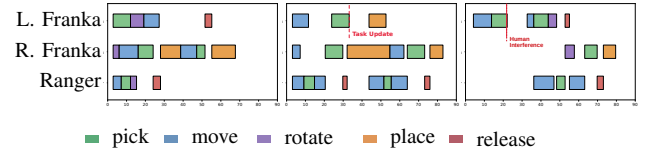


Fig. 10. Hardware timelines for Fig. 8: left, normal motion; middle, task-update revision; right, disturbance recovery.

with the lower event TSR, this result associates selective replanning with lower semantic reasoning occupancy and better task progress under the tested events.

E. Hardware Experiments

The hardware platform contains two fixed Franka manipulators and a third manipulator on an AgileX Ranger Mini V2. An AgileX SCOUT MINI carries task objects as a teleoperated moving carrier; it is not a semantic coordinator. The same shared state, primitive-skill interface, and event update path are used on hardware. Each condition is repeated for 10 trials. Figure 11 exposes the role-level semantic updates for the task-update and disturbance trials. In the top row, the revised instruction enters γ , the affordance role refreshes $\mathcal{F}$, the verification role identifies the affected nodes, and the planning role revises G while retaining unaffected work. The object relocation refreshes $\mathcal{F}$, verification records the failed execution status in z and traces the invalidated dependencies, and planning repairs the affected branch of

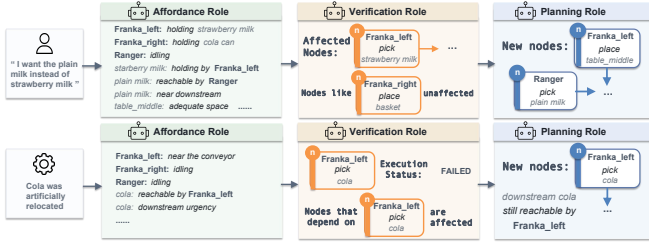


Fig. 11. Intermediate outputs of the affordance, verification, and planning roles after the task-update event in the middle panel of Fig. 10 (**Top**) and the external disturbance in the right panel (**Bottom**).

G. The three role outputs are committed together as the corresponding update of S_{sem} before dispatch resumes.

The three scenarios isolate normal motion, a task update, and a disturbance. Table III reports 10/10 TSR and 100.0% SCR for H1, 9/10 and 98.6% for H2 (11.3 s response, 0.38 affected/pending), and 7/10 and 92.3% for H3 (9.7 s, 0.14). The recorded traces show that semantic updates are committed online before dispatch resumes rather than reconstructed after a run: H2 updates γ after the instruction change, H3 refreshes $\mathcal{F}$ after the disturbance, and verification updates z at execution boundaries. In H3, the resulting revision covers a 14% affected scope while retaining unaffected work. These physical executions therefore corroborate the online update sequence; the lower H3 TSR also shows that selective updating does not eliminate physical uncertainty.

V. CONCLUSION

This work proposes role-shared semantic coordination for dynamic heterogeneous multi-arm manipulation with continual physical changes. The framework assigns explicit semantic responsibilities to the affordance, planning, and verification roles over a shared semantic state, and uses event-driven selective updates to reconsider only the affected scope after task updates, external disturbances, or execution failures. Within the tested heterogeneous manipulation scenarios, baseline, ablation, and hardware results support role separation for semantic responsibility allocation and selective updates for efficient adaptation under semantic events. The framework coordinates semantic decisions and semantic-state updates; it does not replace local skill controllers, perception, low-level control, or physical execution modules. The remaining execution boundary is visible in the 27/30 success rate for simulation execution failures and the 7/10 success rate for the hardware disturbance condition, where physical uncertainty can still prevent completion.

REFERENCES

- [1] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, T. Jackson, N. Brown, L. Luu, S. Levine, K. Hausman, and B. Ichter, "Inner monologue: Embodied reasoning through planning with language models," in *Proceedings of the 6th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 205, 2023, pp. 1769–1782.
- [2] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, "Code as policies: Language model programs for embodied control," in *2023 IEEE International Conference on Robotics and Automation*, 2023, pp. 9493–9500.
- [3] K. Rana, J. Haviland, S. Garg, J. Abou-Chakra, I. Reid, and N. Sünderhauf, "SayPlan: Grounding large language models using 3d scene graphs for scalable robot task planning," in *Proceedings of the 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229, 2023, pp. 23–72.
- [4] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, "VoxPoser: Composible 3d value maps for robotic manipulation with language models," in *Proceedings of the 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229, 2023, pp. 540–562.
- [5] Z. Mandi, S. Jain, and S. Song, "RoCo: Dialectic multi-robot collaboration with large language models," in *2024 IEEE International Conference on Robotics and Automation*, 2024, pp. 286–299.
- [6] K. Liu, Z. Tang, D. Wang, Z. Wang, B. Zhao, and X. Li, "COHERENT: Collaboration of heterogeneous multi-robot system with large language models," in *2025 IEEE International Conference on Robotics and Automation*, 2025.
- [7] M. Tajiri, A. M. Kashani, I. Ahmadi, A. Jadidi, S. Kashani, and B. Khalaj, "MALLVI: A multi-agent framework for integrated generalized robotics manipulation," *arXiv preprint arXiv:2602.16898*, 2026.
- [8] S. Borate, B. R. B. V. Pardeshi, and M. Vadali, "CoMuRoS - an llm-based generalizable hierarchical task planning and execution framework for heterogeneous robot teams with event-driven re-planning," *Frontiers in Robotics and AI*, vol. 13, p. 1843313, 2026.
- [9] Y. Zhu, J. Chen, X. Zhang, M. Guo, and Z. Li, "DEXTER-LLM: Dynamic and explainable coordination of multi-robot systems in unknown environments via large language models," *arXiv preprint arXiv:2508.14387*, 2025.
- [10] S. S. Kannan, V. L. N. Venkatesh, and B.-C. Min, "SMART-LLM: Smart multi-agent robot task planning using large language models," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2024, pp. 12 140–12 147.
- [11] K. Obata, T. Aoki, T. Horii, T. Taniguchi, and T. Nagai, "LiP-LLM: Integrating linear programming and dependency graph with large language models for multi-robot task planning," *IEEE Robotics and Automation Letters*, vol. 10, no. 2, pp. 1122–1129, 2025.
- [12] Y.-X. He, L. Wei, H. Cen, J.-J. Jiang, Z. Li, G. Lu, Y. Yang, D. Zhang, and W.-S. Zheng, "A closed-loop multi-agent framework for robust multi-robot manipulation," in *Robotics: Science and Systems*, 2026.
- [13] N. Marturi, M. Kopicki, A. Rastegarpahan, V. Rajasekaran, M. Adjigble, R. Stolk, A. Leonardis, and Y. Bekiroglu, "Dynamic grasp and trajectory planning for moving objects," *Autonomous Robots*, vol. 43, pp. 1241–1256, 2019.
- [14] E. Maranci, C. A. Avizzano, and S. D'Avella, "Reactive grasp controller for dynamic grasping of moving objects," *Robotics and Autonomous Systems*, vol. 200, p. 105401, 2026.
- [15] J. Liu, R. Zhang, H.-S. Fang, M. Gou, H. Fang, C. Wang, S. Xu, H. Yan, and C. Lu, "Target-referenced reactive grasping for dynamic objects," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 8824–8833.
- [16] J. Haviland, F. Dayoub, and P. Corke, "Control of the final-phase of closed-loop visual grasping using image-based visual servoing," *arXiv preprint arXiv:2001.05650*, 2020.
- [17] O. Pettersson, "Execution monitoring in robotics: A survey," *Robotics and Autonomous Systems*, vol. 53, no. 2, pp. 73–88, 2005.
- [18] M. Brenner and B. Nebel, "Continual planning and acting in dynamic multiagent environments," *Autonomous Agents and Multi-Agent Systems*, vol. 19, pp. 297–331, 2009.
- [19] M. Levihn, L. P. Kaelbling, T. Lozano-Pérez, and M. Stilman, "Fore-sight and reconsideration in hierarchical planning and execution," in *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2013.
- [20] C. Pezzato, C. H. Corbató, S. Bonhof, and M. Wisse, "Active inference and behavior trees for reactive action planning and execution in robotics," *IEEE Transactions on Robotics*, vol. 39, no. 2, pp. 1050–1069, 2023.